Assertions

Einführung

- Programme müssen korrekt funktionieren
- mit Hilfe von Assertions kann man Annahmen über den Zustand des Programms zur Laufzeit überprüfen
- werden hauptsächlich für Testen und Debugging genutzt

Assertions müssen mittels java –ea datapath aktiviert werden

Aufbau

- besteht aus einer Bedingung und einer optionalen Fehlermeldung
  - Wenn eine Bedingung wahr ist, passiert nichts
  - Wenn die Bedingung falsch ist, wird ein AssertionError ausgelöst

```
public class AssertionBeispiel {
    public static void main(String[] args) {
        int alter = 25;

        // Eine einfache Assertion
        assert alter > 0 : "Das Alter muss größer als 0 sein";

        System.out.println("Das Alter ist: " + alter);
    }
}
```

Aufbau

Assertions müssen mittels java –ea datapath aktiviert werden

```
java -ea src/main/java/AssertionBeispiel.java
```

```
class AssertionBeispiel {
    public static void main(String[] args) {
        getQuadrat(-100);
    }

    public static int getQuadrat(int zahl) {
        assert zahl >= 0 : "Die Zahl darf nicht negativ sein";
        return zahl * zahl;
    }
}
```

Aufgabe

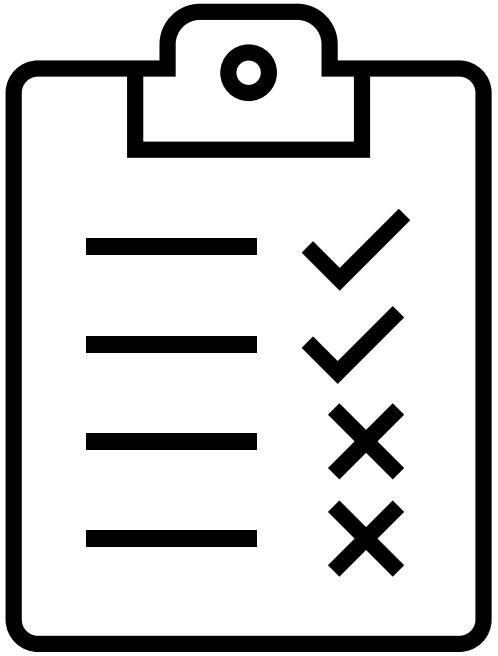
Erstelle eine Methode, die eine Liste von Zahlen erhält und sicherstellt, dass sie nicht leer ist.

Nutze dazu eine Assertion. Schreibe eine Methode checkAlter(int alter), die sicherstellt, dass das Alter positiv ist.

Implementiere eine Methode zur Berechnung der Quadratwurzel mit einer Assertion, die sicherstellt, dass nur nicht-negative Zahlen verwendet werden.

JUnit Tests

Einführung



- JUnit ist ein Framework für automatisierte Tests in Java
- Standard für Testen in Java
- Leicht integrierbar in Maven/Gradle und IDEs

Motivation

- Fehler frühzeitig erkennen
- Sicherstellen, dass Änderungen keine Fehler einführen
- Automatisierte Tests = schnelle Feedbackschleife
- Grundlage um Continuous Integration (CI) sicherer zu gestalten

Aufbau

```
@Test
public void testAddition(){
    assertEquals( expected: 5,  actual: 10 + 5);
}
```

eine Testklasse enthält mehrere Testmethoden

jede Testmethode:

- Ist mit **@Test** annotiert
- Enthält JUnit-**Assertions** (assertEquals, assertTrue,...)

JUnit-Assertions

assertEquals(expected, actual)
assertNotEquals(expected, actual)

assertTrue(condition)
assertFalse(condition)

assertNull(obj)
assertNotNull(obj)

assertThrows(Exception.class, ()->...)

Lebenszyklus-Annotationen

@BeforeAll

einmal vor allen Tests ausgeführt (z.B. Ressourcen starten)

@AfterAll

einmal nach allen Tests (z.B. Ressourcen freigeben)

@BeforeEach

vor jedem Test (z.B. Testobjekte erzeugen)

@AfterEach

nach jedem Test (z.B. Standardwerte zurücksetzen)

Annotationen

@Disabled

@ParameterizedTest
@ValueSource(ints = {1,2,3})

@DisplayName

Best Practice

Tests sind unabhängig voneinander!

Pro Test eine Methode testen

Negative/Positive Testszenarien implementieren

Tests möglichst häufig (automatisiert) ausführen!

Aufgaben

Schreibe einen Test, der überprüft, ob 2 + 3 == 5 gilt.
Implementiere eine Methode int verdopple(int x) und schreibe Tests
Implementiere eine Methode int teile(int a, int b), die bei Division durch 0 eine IllegalArgumentException wirft. Schreibe einen Test mit assertThrows.
Erstelle eine Klasse Rechner mit Methoden add, sub, mul, div. Initialisiere das Objekt **vor jedem Test** mit @BeforeEach.
Schreibe eine Methode, die ein Array sortiert. Teste mit assertArrayEquals, dass [3,1,2] korrekt zu [1,2,3] sortiert wird.

Implementiere eine Methode String findeName(int id), die null zurückgibt, wenn die ID unbekannt ist. Schreibe Tests mit assertNull und assertNotNull.

Erstelle einen parametrisierten Test, der prüft, dass alle Zahlen {1,2,3,4,5} größer als 0 sind.

Schreibe einen Test für eine Methode berechneSteuern(), aber markiere ihn mit @Disabled("Noch nicht implementiert").

Implementiere eine Klasse Person, die Name und Alter speichert. Schreibe einen Test, der mit assertAll gleichzeitig prüft.

boolean istPasswortGueltig(String pw) und Teste auf verschiedene Szenarien. Die üblichen Anforderungen für Passwörter sollen gelten

Quellen

- https://www.vogella.com/tutorials/JUnit/article.html